



CPython

A Complete Guide to CPython's
Architecture and Performance

—
Chien-Lung Kao

Apress®

CPython

A Complete Guide to CPython's Architecture and Performance

Chien-Lung Kao

Apress®

CPython: A Complete Guide to CPython's Architecture and Performance

Chien-Lung Kao
Taipei, T'ai-pei, Taiwan

ISBN-13 (pbk): 979-8-8688-1768-7
<https://doi.org/10.1007/979-8-8688-1769-4>

ISBN-13 (electronic): 979-8-8688-1769-4

Copyright © 2025 by Chien-Lung Kao

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

Trademarked names, logos, and images may appear in this book. Rather than use a trademark symbol with every occurrence of a trademarked name, logo, or image we use the names, logos, and images only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Managing Director, Apress Media LLC: Welmoed Spahr
Acquisitions Editor: Celestin Suresh John
Development Editor: James Markham
Editorial Assistant: Gryffin Winkler

Cover designed by eStudioCalamar

Cover image designed by kuritafsheen77 on freepik

Distributed to the book trade worldwide by Springer Science+Business Media New York, 1 New York Plaza, New York, NY 10004. Phone 1-800-SPRINGER, fax (201) 348-4505, e-mail orders-ny@springer-sbm.com, or visit www.springeronline.com. Apress Media, LLC is a Delaware LLC and the sole member (owner) is Springer Science + Business Media Finance Inc (SSBM Finance Inc). SSBM Finance Inc is a **Delaware** corporation.

For information on translations, please e-mail booktranslations@springernature.com; for reprint, paperback, or audio rights, please e-mail bookpermissions@springernature.com.

Apress titles may be purchased in bulk for academic, corporate, or promotional use. eBook versions and licenses are also available for most titles. For more information, reference our Print and eBook Bulk Sales web page at <http://www.apress.com/bulk-sales>.

Any source code or other supplementary material referenced by the author in this book is available to readers on GitHub. For more detailed information, please visit <https://www.apress.com/gp/services/source-code>.

If disposing of this product, please recycle the paper

Table of Contents

About the Author xi

Chapter 1: Reading the CPython Source Code 1

 What Is CPython? 1

 Why Read the Source Code? 1

 Where to Begin?..... 2

 Obtaining the Source Code 2

 Development Tools 3

 Can I Understand This Without Knowing C? 4

Chapter 2: Overview of the CPython Project Structure..... 5

 Project Structure..... 5

 Building the Project..... 7

 Greet CPython!..... 8

 A (Very) Basic Module..... 11

 Saying Goodbye on Exit..... 13

Chapter 3: Everything Is an Object: Part 1 15

 What Is an “Object”? 15

 The Previous and Next Object..... 17

 Garbage Collection Mechanism 18

 PyObject 22

 Summary..... 24

TABLE OF CONTENTS

Chapter 4: How Objects Are Created in CPython 25

Running the Program! 25

 Step 0: Code Analysis 25

 Step 1: Transformation to AST 26

 Step 2: Compilation to Bytecode 27

 Step 3: Instantiating an Object 29

Summary..... 31

Chapter 5: Everything Is an Object: Part 2..... 33

PyTypeObject..... 33

 Basic Members..... 34

 Methods and Operators 34

 Access Methods 35

 Other 35

The List Type 36

 Printing a List 37

 Using the Bracket Operator 39

 List Methods 41

 List Addition..... 42

 Number of Elements..... 44

 List Comparison..... 45

Summary..... 47

Chapter 6: Defining a Custom Built-in Type..... 49

Creating a New Type 49

 Defining Methods 50

 Implementing the Type 51

Making It a Built-in Type 53

Building and Running..... 54

Parameterized Initialization 55

Chapter 7: What Happens During Module Import	61
Different Ways of Importing	61
The import Instruction	62
The from .. import .. Instruction.....	66
Crazy Side-Effects!	67
The Behind-the-Scenes Hero: meta_path	69
Summary.....	71
Chapter 8: The Internal Representation of Integers	73
How Are Numbers Created?.....	73
Integer Objects.....	75
Astronomical Numbers!.....	77
Small Integers	78
Chapter 9: Floating Point Numbers in CPython	81
What Is a “Floating Point Number”?	81
The Structure of Floating Point Numbers.....	82
About Floating Point Numbers	83
Floating Point Arithmetic	84
Infinity!	86
Not a Number!	87
Comparing Floating Point Numbers	88
Floating Point Performance	89
Chapter 10: Inside the String Object: Part 1	91
Creating a String	91
String Objects.....	92
The Fundamental String Structure	95
String Operations	96
Encoding Conversion	96
Strings Are Immutable.....	98

TABLE OF CONTENTS

Chapter 11: Inside the String Object: Part 2 101

String Operations 101

 Copying Strings 101

 String Slicing 104

Performance 107

 String Interning..... 107

Chapter 12: What Happens When Python Starts..... 111

Using a Debugger..... 111

Program Entry Point..... 112

Reading the Program File 115

Building the Abstract Syntax Tree 117

Creating the Code Object 118

Ready for Liftoff! 119

Chapter 13: From Source to Bytecode: How .py Becomes .pyc 121

Having a .pyc Is All You Need 121

“Maybe” a .pyc File?..... 123

Magic Number, Magic! 125

Unpacking a .pyc File..... 129

Chapter 14: The List Object and Its Internal Management 133

Internal Structure of Lists 133

 Creating and Initializing a List 134

Memory Management..... 137

 When Is More Memory Needed? 138

 The Over-allocation Formula 140

Common List Operations..... 141

 Appending Elements..... 141

 Inserting Elements..... 142

 Removing Elements..... 143

Chapter 15: The Dictionary Object: Part 1	145
The Internal Structure of Dictionaries.....	145
Creating a Dictionary	147
Adding Elements.....	149
Handling Hash Collisions	154
Looking Up Elements.....	157
Chapter 16: The Dictionary Object: Part 2	159
Dictionary Memory Management Techniques.....	159
Adding More Elements	159
Should We Request More Capacity?.....	162
How Much Space to Allocate?	166
Returning Memory: Does It Happen?.....	167
Chapter 17: The Tuple Object and Its Immutability	169
Tuple Design.....	169
Creating a Tuple	170
Empty Tuples	172
Non-empty Tuples.....	173
Deallocation Mechanism	174
Common Tuple Operations	177
Modifying Tuples.....	177
Tuple Unpacking	178
Chapter 18: Inside the Python VM: Code Objects	181
Functions Are Also Objects.....	181
Preparing to Create a Function.....	184
Code Object	185

TABLE OF CONTENTS

Chapter 19: Inside the Python VM: Function Objects 191

 Creating Function Objects..... 191

 What Do the Parameters Look Like? 193

 Accessing Function Attributes 194

 Calling a Function 195

 What Is “Vectorcall”?..... 196

Chapter 20: Inside the Python VM: Frame Objects 199

 Frame Object..... 199

 The Life Cycle of a Frame Object 201

Chapter 21: Inside the Python VM: Namespaces and Scopes 205

 Variable Scope 205

 Local Variables (L) 206

 Global and Built-in Variables (G, B) 207

 Enclosing Variables (E) 211

Chapter 22: Inside the Python VM: Cells and Closures..... 215

 Creating a Cell Object 215

 Closures..... 217

 Free Variables..... 219

 From the Python Perspective 220

Chapter 23: Classes and Where They Come From 223

 Creating a Class 223

 The Mastermind Behind the Scenes 226

 Selecting the Metaclass 227

 Preparing the Namespace 228

 The Birth of a Class! 229

 Chicken or the Egg? 231

Chapter 24: Class Inheritance in CPython	233
Classes and Inheritance.....	233
Creating a Class.....	233
How Does Inheritance Work?	235
Method Lookup.....	236
Chapter 25: Method Resolution Order and C3 Linearization.....	243
The C3 Linearization Algorithm	243
Whose Method Gets Called?.....	244
MRO Calculation.....	245
Single Inheritance.....	245
Multiple Inheritance.....	249
A More Complex Inheritance Example.....	250
What If It Can't Be Calculated?	253
Chapter 26: The Role of super() in Multiple Inheritance.....	257
Algorithm Implementation	257
Preparation Before Merging	257
Merging	260
Family Feuds.....	262
Super!	263
Whose Child Is It?	265
Solving Family Feuds.....	269
Specifying the Superclass	270
Quick Quiz: Who Am I?	271
Chapter 27: The Generator Object and the Yield Statement	273
The Generator Class.....	274
yield, Please!	278
Next, Please!.....	279

TABLE OF CONTENTS

Chapter 28: How Iterators Work Internally 283

 The Iterator Protocol 283

 Halt! Password, Please!..... 285

 Different Types of Iterators? 291

Chapter 29: Understanding Descriptors in Python 295

 When Calling Methods 295

 Attribute Lookup Process 296

 Process Summary 300

 Method Descriptors..... 301

Chapter 30: Exception Handling Internals in CPython 305

 Exception Handling 305

 Stacking Up 306

 Exception Table..... 307

 Entry Portals 308

 Exception Type Matching..... 308

 Handling the Exception..... 310

 Finally! 311

Index..... 313

About the Author

Chien-Lung Kao is a seasoned programmer with over 30 years of hands-on experience and 15 years as an educator, specializing in JavaScript, Ruby, and Python. He is a respected author of several books on Ruby on Rails and Git published in Taiwan, sharing his in-depth knowledge and insights with a global audience. Kao has also been a featured speaker at conferences and tech events across Asia, including Taiwan and Japan, where he connects with developers and tech enthusiasts on advanced programming techniques and industry trends.

CHAPTER 1

Reading the CPython Source Code

What Is CPython?

If this is your first time hearing the name “CPython,” you might think it’s a new implementation of Python. But no, CPython isn’t a new toy or a new programming language—it’s the very same Python interpreter that most people use every day. What some may not know is that Python was originally developed in the C programming language, so when people mention “Python” without any further clarification, they are typically referring to CPython.

Why Read the Source Code?

Why? For no particular reason—just for fun. And having fun is important!

If your goal in reading this book is to dramatically improve your Python programming skills by exploring the CPython source code, you might be a little disappointed. While reading the CPython source can enhance your understanding of Python to some degree, the primary benefit is the opportunity to improve your C programming skills, since most of the CPython code base is written in C.

Therefore, if you simply want to learn Python syntax, an introductory Python book would be a better choice. The main purpose of delving into the CPython source code is to uncover some of the “black magic” behind the language. For example, what exactly are “objects” in Python? How are modules loaded? How does Python manage memory? These are the kinds of topics this book will focus on.

Where to Begin?

Obtaining the Source Code

Although the entire CPython project is available to browse directly on GitHub, it's more efficient to download the CPython source code onto your local computer for in-depth exploration. The next chapter will guide you through where and how to download it.

While the latest version of Python is currently 3.13, this book references the CPython source code for version 3.12.6. Source code can vary between different versions, especially for changes beyond minor releases. If you want to follow along exactly, it's recommended to use the same version as I do to ensure that we're viewing the same content.

Additionally, because the CPython source code can be quite complex at times, I may occasionally omit sections for clarity. For example, the original code might look like this:

```
// file: Include/object.h

struct _object {
    PyObject_HEAD_EXTRA

    #if (defined(__GNUC__) || defined(__clang__)) \
        && !(defined __STDC_VERSION__ && __STDC_VERSION__ >= 201112L)
        // On C99 and older, anonymous union is a GCC and clang extension
        __extension__
    #endif

    #ifdef _MSC_VER
        // Ignore MSC warning C4201: "nonstandard extension used:
        // nameless struct/union"
        __pragma(warning(push))
        __pragma(warning(disable: 4201))
    #endif

    union {
        Py_ssize_t ob_refcnt;
    #if SIZEOF_VOID_P > 4
        PY_UINT32_T ob_refcnt_split[2];
    #endif
    };
};
```

```

#ifdef _MSC_VER
    __pragma(warning(pop))
#endif

    PyTypeObject *ob_type;
};

```

There are several `#ifdef` conditional compilation directives, some related to the operating system, others to debugging information. Most of the time, these may not be directly relevant to the core topic at hand, so for the sake of focus, I might simplify the code like this:

```

// file: Include/object.h

struct _object {
    _PyObject_HEAD_EXTRA

    union {
        Py_ssize_t ob_refcnt;
    };

    PyTypeObject *ob_type;
};

```

To be clear, it's not that the omitted portions are unimportant, but removing them helps us concentrate on the essentials.

Development Tools

There are many tools you can use to read source code, such as Vim, Visual Studio Code (hereafter referred to as VS Code), or even full-featured IDEs like Visual Studio. These editors provide helpful features for source code navigation, such as jumping quickly to definitions of functions, macros, constants, or searching for keywords—all of which make source code exploration more efficient.

In this book, I will use VS Code for source code exploration. If you haven't installed it yet, you can download it from the [official VS Code website](https://code.visualstudio.com/).

Besides VS Code, I have also started using a new editor called [Zed](https://zed.dev/), which is developed in Rust. While it doesn't have as many plugins as VS Code, its performance is significantly better, especially for larger files or projects with numerous lines of code—this difference is noticeable. I highly recommend trying it out.

- **VS Code:** <https://code.visualstudio.com/>
- **Zed:** <https://zed.dev/>

Can I Understand This Without Knowing C?

Here's the bottom line: Even if you don't grasp everything 100%, it's perfectly fine. Skimming through the code can still be a valuable learning experience.

Learning is not always linear; you don't need to understand every single line to gain insights. Sometimes, just examining code structure, function calls, or variable names can be informative. It's like piecing together a puzzle—you don't have to complete every piece to get a sense of the overall picture.

To be honest, I'm not a C expert either. Fortunately, we live in an age of advanced AI tools such as ChatGPT that can help explain what's going on when things get confusing. As I go through the source code, I'm learning C along the way too. Even if it feels like “the blind leading the blind,” you'll eventually get a sense of the bigger picture.

Also, not every part of the CPython project is written in C—some modules are written in Python. If you're already comfortable with Python, you might want to start by looking at those modules, which tend to be more approachable. However, if you're a complete beginner and have never written any kind of code before, you might find it challenging to keep up. For this reason, I assume readers have some basic programming knowledge—it doesn't have to be Python; any language will do—as long as you're familiar with concepts like variables, functions, loops, and control flow.

Ready to get started? In the next chapter, we'll take a look at the CPython project structure and walk through compiling the source code on your own machine, so you can build your very own version of Python!

CHAPTER 2

Overview of the CPython Project Structure

In this chapter, we'll take a look at the overall structure of the CPython project and try building the project ourselves. We'll even make some small modifications to the CPython source code to get a taste of what it's like to be a Python Core Developer (just for fun, of course)!

Project Structure

First, let's grab a copy of the project from GitHub:

```
$ git clone git@github.com:python/cpython.git -b v3.12.6 --depth=1
```

In this book, I am using version 3.12.6 of CPython. If you're using a different version, some source code details might vary. Excluding some less important files, the directory structure of the freshly cloned CPython project from GitHub looks like this:

```
CPython
|— Doc
|— Grammar
|— Include
|— Lib
|— Mac
|— Misc
|— Modules
|— Objects
|— PC
|— PCbuild
```

```
|— Parser
|— Programs
|— Python
|— Tools
```

Here's a brief introduction to what each directory contains:

- **Doc:** As the name suggests, this directory contains all the documentation, written in reStructuredText (.rst) format. It's actually pretty good bedtime reading if you're struggling to fall asleep!
- **Grammar:** Files for defining how Python's grammar and parser rules work.
- **Include:** C language header files used throughout the project. If you plan to write extensions for CPython, you'll probably need files from here.
- **Lib:** The standard library—this directory contains modules written in Python. If you're familiar with Python, you'll find this folder relatively approachable.
- **Modules:** Similar to the Lib directory, but the modules here are written in C.
- **Mac:** Tools and files specific to macOS.
- **Misc:** A collection of miscellaneous files. I personally use such folders to store files that I can't categorize elsewhere.
- **Objects:** The source code for all Python built-in object types, such as str or list. You'll see files from this directory quite often in this book.
- **PCbuild:** Files for building Python on Windows, especially with Visual Studio. There are project files you can open directly.
- **PC:** Similar to PCbuild, but targeted at older Windows versions. Most files are obsolete, but some are retained for compatibility.
- **Parser:** Code for converting .py files into tokens that Python can understand. This part is a bit more advanced.

- **Programs:** Contains source code related to CPython executables.
- **Python:** Contains the interpreter source code itself. This is a more advanced area, but if you're interested in how the interpreter works, it's worth checking out.
- **Tools:** Utilities for Python development and maintenance.

For the purposes of this book, you'll most often see the `Include`, `Lib`, `Modules`, `Objects`, and `Python` directories. These are the core source files of the CPython interpreter, and if you want to understand the inner workings of Python, you'll spend most of your time here.

Building the Project

After downloading the project, switch into the project directory and run the `./configure` command:

```
$ cd cpython
$ ./configure
```

The `./configure` command will print out a lot of information that might look intimidating at first. What it's really doing is checking your system environment and dependencies to make sure nothing important is missing. Once everything checks out, a `Makefile` will be generated. This file describes how the `make` command should go about compiling the whole project.

If you add the `--prefix` parameter to `./configure`, like so:

```
$ ./configure --prefix=/tmp/my-python
```

With the `--prefix` option, when you run `make install` later, the compiled Python (and related files) will be installed under `/tmp/my-python`. However, I'm not planning to run `make install` just yet, so you can leave off the `--prefix` for now. We'll revisit it if we need to run other tools (like `pip`) later.

Next, run the `make` command. This command will compile the whole project according to the rules defined in the generated `Makefile`:

```
$ make
```

The compilation process may take a while. If it finishes without any errors, you should find a new executable named `python.exe` in the root directory—even on macOS, the filename is `python.exe`. I know it might seem odd to see `.exe` on macOS, but it’s intentional: since there’s already a `Python/` directory in the CPython project, using `python.exe` helps avoid a naming conflict.

If you want to “install” the Python version you just built, you can run:

```
$ make install
```

If you passed a `--prefix` argument to `./configure` earlier, the `make install` command will install Python to that location. But even if you skip installation, you can still directly run the newly built `python.exe`. Try running it to launch the familiar REPL environment:

```
$ ./python.exe
Python 3.12.6+ (heads/3.12:b2a7d718e3b, Sep 15 2024, 23:31:57) [Clang
15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import sys
>>> sys.version
'3.12.6+ (heads/3.12:b2a7d718e3b, Sep 15 2024, 23:31:57) [Clang 15.0.0
(clang-1500.3.9.4)]'
```

A `+` after the version number indicates that this is not an official release, but rather a “development” or custom-compiled version.

Greet CPython!

Now, let’s make a small change to the CPython source code to create a custom effect. For example, I’d like to print a “Hello” message upon entering the REPL and a “Bye” message when exiting, just to make things a bit friendlier! To achieve this, I need to locate the code responsible for entering the REPL. This is in `Python/pythonrun.c`, and by looking up the `_PyRun_InteractiveLoopObject()` function, you’ll find a `do...while...` loop—the very loop that implements the “Read-Eval-Print Loop” (REPL):

```
// file: Python/pythonrun.c

int
_PyRun_InteractiveLoopObject(FILE *fp, PyObject *filename,
PyCompilerFlags *flags)
{
    PyCompilerFlags local_flags = _PyCompilerFlags_INIT;
    // ... omitted ...

    do {
        ret = PyRun_InteractiveOneObjectEx(fp, filename, flags);
        // ... omitted ...
    } while (ret != E_EOF);
    return err;
}
```

This segment is fairly straightforward—the main logic lives within the `do... while...` loop. If I want to greet the user before the REPL loop starts, I can simply call `printf()` right before `do`. To look a bit more “programmatic,” I’ll define a `say_something()` function in this file, which simply prints the passed-in string:

```
// file: Python/pythonrun.c

void
say_something(const char *message)
{
    printf("=====\n");
    printf("%s\n", message);
    printf("=====\n");
}
```

Because we’ll call `say_something()` inside `_PyRun_InteractiveLoopObject()`, the function definition must appear before its first use. Now, let’s call it:

```
// file: Python/pythonrun.c

int
_PyRun_InteractiveLoopObject(FILE *fp, PyObject *filename,
PyCompilerFlags *flags)
```

```

{
    PyCompilerFlags local_flags = _PyCompilerFlags_INIT;
    // ... omitted ...

    say_something("Hello CPython"); // added this line

    do {
        ret = PyRun_InteractiveOneObjectEx(fp, filename, flags);
        // ... omitted ...
    } while (ret != E_EOF);

    say_something("Bye"); // added this line
    return err;
}

```

This way, “Hello CPython” will print when you enter the REPL, and “Bye” will print upon exit. Unlike Python or JavaScript, where changes take effect immediately, with C, you need to recompile the code first. So re-run make to rebuild CPython. This time, only the changed files are rebuilt, so it should finish faster than before.

Run the following to see the effect:

```

$ ./python.exe
Python 3.12.6+ (heads/3.12-dirty:b2a7d718e3b, Sep 16 2024, 14:46:05) [Clang
15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
=====
Hello CPython
=====
>>> 1 + 2
3
>>> print("Hey you!")
Hey you!
>>> ^D
=====
Bye
=====

```

As soon as you enter the REPL, you'll see "Hello CPython". When you exit (by pressing Ctrl+D), "Bye" is printed as well. Success!

However, there is a minor issue. In Python 3.12, you can exit the REPL not only with Ctrl+D but also by typing `exit()`. Currently, `exit()` does not print "Bye", since the original `do...while...` loop condition only checks for `ret != E_EOF`. We'll tackle this shortly, but first, let's improve the `say_something()` function a bit. I want to make it reusable in other parts of the code base, so I'll reorganize it into a separate module. This will also demonstrate how to structure modules in C!

A (Very) Basic Module

To define a module in C, you typically declare function prototypes in a `.h` header file, then implement the functionality in a corresponding `.c` source file. This is common practice in C. Following the conventions in the CPython project, such header files are usually placed in the Include directory and source files in Modules, Python, or Objects. Extension modules (C Extensions) are typically in Modules, while core features are found in Python or Objects. Since our `say_something()` function modifies interpreter behavior, placing it in the Python directory makes sense.

Let's start by creating a `greeting.h` file in the Include directory. The filename can be something else if you prefer. The contents are as follows:

```
// file: Include/greeting.h

#ifndef _PY_GREETING_H
#define _PY_GREETING_H

extern void say_something(const char *message);

#endif
```

The `#ifndef` and `#define` lines implement a header guard, which prevents this file from being included more than once and causing compilation errors. `_PY_GREETING_H` is just an arbitrary name; the key is that it doesn't clash with any others. Next, in the Python directory, create a `greeting.c` file:

```
// file: Python/greeting.c

#include <stdio.h>
#include "greeting.h"
```

```
void say_something(const char *message)
{
    printf("=====\n");
    printf("%s\n", message); // Print the incoming message
    printf("=====\n");
}
```

This is functionally identical to before, but now we include "greeting.h" so the compiler knows about say_something().

Now we need to tell CPython to compile these files so that say_something() is available. In the CPython project, all files to be compiled are listed in Makefile.pre.in, which serves as a template for generating the Makefile. Let's add greeting.c to the build by finding the PYTHON_OBJS list and inserting our greeting object file:

```
# file: Makefile.pre.in

PYTHON_OBJS= \
    Python/_warnings.o \
    ... omitted ...
    Python/suggestions.o \
    Python/perf_trampoline.o \
    Python/greeting.o \
    Python/$(DYNLOADFILE) \
    $(LIBOBJS) \
    $(MACHDEP_OBJS) \
    $(DTRACE_OBJS) \
    @PLATFORM_OBJS@
```

After saving the file, you need to re-run ./configure to regenerate the Makefile. Once that's complete, run make again to rebuild CPython. This will compile greeting.c into greeting.o. Now, anywhere you'd like to use this function, just include greeting.h, for example:

```
// file: Python/pythonrun.c

#include "pycore_pylifecycle.h" // _Py_UnhandledKeyboardInterrupt
#include "pycore_pystate.h" // _PyInterpreterState_GET()
#include "pycore_sysmodule.h" // _PySys_Audit()
```



```
#include "pycore_traceback.h"    // _PyTraceBack_Print_Indented()
#include "greeting.h"

// ... omitted ...
```

As long as you include `greeting.h`, you can call your custom `say_something()` function wherever you like.

Saying Goodbye on Exit

Back to the REPL: earlier, we mentioned that quitting with `exit()` doesn't print "Bye", because calling `exit()` does not trigger the `ret != E_EOF` condition in the loop. In `Python/pythonrun.c`, search for the `handle_system_exit()` function. This is the function that's called when `exit()` is used to leave the REPL. Let's add a call to `say_something("Bye");` here:

```
// file: Python/pythonrun.c

static void
handle_system_exit(void)
{
    int exitcode;
    if (_Py_HandleSystemExit(&exitcode)) {
        say_something("Bye"); // <-- add this line
        Py_Exit(exitcode);
    }
}
```

Rebuild with `make`, and it should work as expected:

```
$ ./python.exe
Python 3.12.6+ (heads/3.12-dirty:b2a7d718e3b, Sep 16 2024, 15:41:49) [Clang
15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
=====
Hello CPython
=====
>>> print("Hey")
```

CHAPTER 2 OVERVIEW OF THE CPYTHON PROJECT STRUCTURE

Hey

```
>>> exit()
```

=====

Bye

=====

Done! How does it feel to make your own modifications to the CPython source code?

When learning Python, you may have heard the saying “everything in Python is an object.” In the next chapter, let’s start from this idea and take a look at what these “objects” actually look like under the hood!

CHAPTER 3

Everything Is an Object: Part 1

In most Python courses, articles, or tutorial videos, you often hear the statement “everything in Python is an object.” Integers, floats, and strings are all objects; lists and dictionaries obviously look like objects; and even functions and classes are also objects. Since everything is an object, let’s take a look at what an object actually looks like in CPython.

What Is an “Object”?

In CPython, objects are represented using the `PyObject` struct. Let’s see how `PyObject` is defined. In the `Include/pytypedefs.h` file, you’ll find:

```
// file: Include/pytypedefs.h

// ... omitted ...
typedef struct PyGetSetDef PyGetSetDef;
typedef struct PyMemberDef PyMemberDef;

typedef struct _object PyObject;
typedef struct _longobject PyLongObject;
typedef struct _typeobject PyTypeObject;
typedef struct PyCodeObject PyCodeObject;
// ... omitted ...
```

As expected, PyObject is indeed a struct. If you look up `_object`, you'll find its definition in `Include/object.h`:

```
// file: Include/object.h

struct _object {
    PyObject_HEAD_EXTRA

    #if (defined(__GNUC__) || defined(__clang__)) \
        && !(defined __STDC_VERSION__ && __STDC_VERSION__ >= 201112L)
        // On C99 and older, anonymous union is a GCC and clang extension
        __extension__
    #endif

    #ifndef _MSC_VER
        // Ignore MSC warning C4201: "nonstandard extension used:
        // nameless struct/union"
        __pragma(warning(push))
        __pragma(warning(disable: 4201))
    #endif

        union {
            Py_ssize_t ob_refcnt;
            #if SIZEOF_VOID_P > 4
                PY_UINT32_T ob_refcnt_split[2];
            #endif
        };

    #ifndef _MSC_VER
        __pragma(warning(pop))
    #endif

    PyTypeObject *ob_type;
};
```

This looks a bit complicated, so let's remove the conditional compilation directives like `#ifdef` to focus on the core structure. What remains is:

```
// file: Include/object.h

struct _object {
    PyObject_HEAD_EXTRA
```

```

union {
    Py_ssize_t ob_refcnt;
};

PyTypeObject *ob_type;
};

```

The `_object` struct is quite straightforward—it contains a macro `_PyObject_HEAD_EXTRA` and two member variables, `ob_refcnt` and `ob_type`. Let’s examine each of them one by one.

The Previous and Next Object

Starting with the macro `_PyObject_HEAD_EXTRA`, if you trace its definition, you’ll find it in the same file:

```

// file: Include/object.h

#ifdef Py_TRACE_REFS
/* Define pointers to support a doubly-linked list of all live heap
objects. */
#define _PyObject_HEAD_EXTRA          \
    PyObject *_ob_next;               \
    PyObject *_ob_prev;

#define _PyObject_EXTRA_INIT _Py_NULL, _Py_NULL,

#else
# define _PyObject_HEAD_EXTRA
# define _PyObject_EXTRA_INIT
#endif

```

In C, `#define` is used to create macros. Macros are neither variables nor functions—they are code snippets that are replaced during compilation, making the code more readable and reusable. Notice the comment just above the macro:

Define pointers to support a doubly linked list of all live heap objects.

This means that the `_PyObject_HEAD_EXTRA` macro actually defines `_ob_next` and `_ob_prev` pointers, which are used to link all Python objects together into a doubly linked list. Python objects are stored on the heap, which, unlike the stack, does not necessarily occupy contiguous memory locations. After allocation, objects may be scattered throughout the heap. So how does Python know what the previous or next object is? Python uses a “doubly linked list” data structure to connect all objects. This makes it easy to find the previous or next object and allows objects to be added or removed efficiently ($O(1)$), though searching through the list is slower ($O(n)$).

From the source code, you can see that the `_PyObject_HEAD_EXTRA` macro is typically empty during normal compilation and only includes the `_ob_next` and `_ob_prev` pointers when `Py_TRACE_REFS` is defined. This feature is usually enabled for debugging or tracking object reference counts; in the Python versions most of us use, objects do not form a doubly linked list.

Garbage Collection Mechanism

`ob_refcnt` is a variable of type `Py_ssize_t`, which is essentially a long integer. `refcnt` stands for Reference Count (RC), which is how Python handles garbage collection. When an object is referenced, such as being assigned to a variable or stored in a list or dictionary, its `ob_refcnt` increases. Conversely, when the reference is removed, `ob_refcnt` decreases. When the value of `ob_refcnt` drops to 0, it means no one needs this object anymore, and it can be reclaimed by the Garbage Collector (GC), releasing its resources back to the system.

Here, `ob_refcnt` is encapsulated in a union; this is usually done to allow for extension or modification of struct fields under different compilation conditions. Let’s trace how `ob_refcnt` is initialized:

```
// file: Objects/object.c
```

```
PyObject *
_PyObject_New(PyTypeObject *tp)
{
    PyObject *op = (PyObject *) PyObject_Malloc(_PyObject_SIZE(tp));
    if (op == NULL) {
        return PyErr_NoMemory();
    }
}
```